ReviewMind

Build Phases & Progress Guide

Phase 1 Foundation	**Phase 2** GitHub Client	**Phase 3** Diff Engine	**Phase 4** Security Engine
Phase 5 Style Engine	**Phase 6** MCP + Claude API	**Phase 7** Frontend	**Phase 8** A2A Agents

Current Status: Phases 1 & 2 DONE - Phase 3 is NEXT

Generated: June 25, 2026

Project Progress Overview

Build Timeline

Phase	Name	Timeline	Status	Key Deliverable
Phase 1	Foundation	Days 1-2	Done	docker compose up + /health + ping tool
Phase 2	GitHub Client + Cache	Days 3-4	Done	httpx client, Redis cache-aside, rate limiter - 12 tests green
Phase 3	Semantic Diff Engine	Days 5-6	Next	unidiff parsing, AST chunking, classification
Phase 4	Security Engine	Day 7	Upcoming	5 rules: SEC001-005, rule registry
Phase 5	Style Engine	Day 8	Upcoming	reviewmind.yaml, 6 style rules
Phase 6	MCP + Claude API	Days 9-10	Upcoming	19 tools wired, /chat endpoint, auth gate
Phase 7	Frontend	Days 11-14	Upcoming	Next.js chat UI, paste, upload, auth modal
Phase 8	A2A Agents	Post-MVP	Future	Parallel sub-agents, orchestrator

Visual Progress (2 of 8 Phases Complete)

What Exists in the Repo Today

File / Item	What It Does
docker-compose.yml	Postgres + Redis services - run with: docker compose up -d
pyproject.toml	All Python dependencies declared
.env / .env.example	Environment config - copy .env.example to .env and fill in
reviewmind/core/config.py	Pydantic Settings - reads GITHUB_TOKEN, DATABASE_URL, REDIS_URL
reviewmind/core/logging.py	structlog configured for JSON output
reviewmind/core/cache.py	Redis asyncio client + cache_aside helper + ETag key builders
reviewmind/db/models.py	5 SQLAlchemy models: repositories, pull_requests, diff_chunks, findings, reviews
reviewmind/db/session.py	AsyncSession factory + get_session FastAPI dependency
reviewmind/api/main.py	FastAPI app with lifespan startup/shutdown
reviewmind/api/routes.py	/health endpoint - verifies DB + Redis
reviewmind/mcp_server.py	MCP server with ping tool (placeholder - real tools in Phase 6)
alembic/versions/108d2*.py	First migration - health check table
alembic/versions/0002_*.py	Full schema - 5 tables with indexes
services/github/client.py	GitHubClient: httpx AsyncClient, auth, ETags, pagination, error mapping
services/github/rate_limiter.py	get_with_retry: exponential backoff on 429/403, warn < 100 remaining
services/github/repository.py	GitHubRepository: get_pr, get_pr_diff, get_pr_files with Redis cache-aside
tests/fixtures/github/*.json	pr.json + pr_files.json fixture data for mocked tests
tests/test_github_client.py	12 tests all green - client, repository, ETag, paginate
scripts/generate_pdf.py	Project overview PDF generator
scripts/generate_phases_pdf.py	This file - build phases PDF generator

Full Project File Tree - Status per File

Every file the project will have at completion. Green = exists and working. Blue = next to build. Gray = future phases.

reviewmind/		
__init__.py	Done	Package init
mcp_server.py	Done	MCP server - ping tool only, 8 real tools pending
core/		
config.py	Done	Pydantic settings - loads from .env
logging.py	Done	structlog JSON logging
cache.py	Done	Redis async client + cache_aside helper
api/		
main.py	Done	FastAPI app, CORS, lifespan
routes.py	Done	/health endpoint
static/index.html	Done	Dev console UI
db/		
models.py	Done	SQLAlchemy models - 5 tables
session.py	Done	Async session factory
services/		
github/	Done	Phase 2 complete
__init__.py	Done	Package init
client.py	Done	httpx async client, auth, ETags, pagination
rate_limiter.py	Done	Rate limit awareness + exponential backoff
repository.py	Done	get_pr, get_pr_diff, get_pr_files + ETag cache
engine/		
diff_parser.py	Pending	Phase 3 - unidiff parsing
chunker.py	Pending	Phase 3 - AST semantic chunking
classifier.py	Pending	Phase 3 - change classification
complexity.py	Pending	Phase 3 - cyclomatic complexity
security/	Pending	Phase 4
rules.py	Pending	5 security rules (SEC001-005)
engine.py	Pending	Rule registry + aggregation
style/	Pending	Phase 5
rules.py	Pending	Style rules (STY001-006)
engine.py	Pending	reviewmind.yaml loader + runner
alembic/		
versions/		
108d2ae2ccf2_*.py	Done	Initial migration - health check table
0002_full_schema.py	Done	5 core tables - repositories, pull_requests, diff_chunks, findings, reviews
tests/		
__init__.py	Done	
fixtures/		
github/		
pr.json	Done	PR #42 fixture - number, title, author, state
pr_files.json	Done	2-file fixture - auth.py added, routes.py modified
test_github_client.py	Done	12 tests passing - respx mocks, ETag, paginate
test_diff_parser.py	Pending	Phase 3 - fixture diffs
test_security_rules.py	Pending	Phase 4 - pos/neg/FP per rule
test_style_rules.py	Pending	Phase 5

test_e2e_golden_path.py	**Pending**	*Phase 6 - full flow*
scripts/		
generate_pdf.py	**Done**	*Project overview PDF generator*
generate_phases_pdf.py	**Done**	*This file - build phases PDF*
docker-compose.yml	**Done**	*Postgres + Redis*
pyproject.toml	**Done**	*Dependencies, build config*
.env / .env.example	**Done**	*Secrets (never commit .env)*

Done - exists and working

Pending - to be built

Dir - directory

Phase 1 - Foundation (Days 1-2)

Done

Objective

Stand up the project skeleton: Docker Compose for Postgres + Redis, FastAPI with a /health endpoint that verifies both services, SQLAlchemy async models for all 5 tables, Alembic migration, structured logging, Redis client, and a bare MCP server with a ping tool.

Files to Create

- reviewmind/core/config.py - Pydantic settings, loads .env
- reviewmind/core/logging.py - structlog JSON logging + correlation ID
- reviewmind/core/cache.py - Redis asyncio client
- reviewmind/db/models.py - 5 SQLAlchemy models
- reviewmind/db/session.py - AsyncSession factory
- reviewmind/api/main.py - FastAPI app + lifespan
- reviewmind/api/routes.py - /health route
- reviewmind/mcp_server.py - MCP server + ping tool
- alembic/versions/0001_*.py - Initial migration
- docker-compose.yml - Postgres + Redis services
- pyproject.toml - Project deps + build config

Step-by-Step Build Tasks

1	Install fpdf2, mcp, fastapi, sqlalchemy, redis, structlog, alembic
2	Write docker-compose.yml with postgres + redis services
3	Create core/config.py - Settings class reading from .env
4	Create core/logging.py - configure structlog for JSON output
5	Create core/cache.py - async Redis client with ping test
6	Write all 5 SQLAlchemy models (repositories, pull_requests, diff_chunks, findings, reviews)
7	Create db/session.py with AsyncSession and get_session dependency
8	Run: alembic init alembic && alembic revision --autogenerate
9	Run: alembic upgrade head - verify tables exist in Postgres
10	Write FastAPI app with /health route checking DB + Redis
11	Write mcp_server.py with ping tool - verify Claude Desktop connects
12	Run: docker compose up && curl localhost:8000/health -> {status: ok}

Definition of Done

[OK] docker compose up starts without errors

[OK] GET /health returns {status:ok, database:ok, redis:ok}

[OK] alembic upgrade head creates all 5 tables

[OK] MCP ping tool responds pong from Claude Desktop

[OK] JSON logs appear in stdout with correlation_id field

Verify with: docker compose up -d && curl localhost:8000/health

Phase 2 - GitHub Client + Redis Cache (Days 3-4)

Done

Objective

Build the GitHub integration layer: a low-level HTTP client with auth/ETags, a rate-limit awareness module, and a high-level repository service. Wrap all GitHub calls with Redis cache-aside so repeated calls hit cache not GitHub.

Files to Create

- reviewmind/services/github/__init__.py
- reviewmind/services/github/client.py - httpx client, auth, ETags, pagination
- reviewmind/services/github/rate_limiter.py - X-RateLimit-* header tracking + backoff
- reviewmind/services/github/repository.py - get_pr, get_pr_diff, get_pr_files
- alembic/versions/0002_full_schema.py - add all 5 tables properly
- tests/fixtures/github/pr_42.json - real response captured + sanitized
- tests/test_github_client.py - mocked with respx

Step-by-Step Build Tasks

1	pip install httpx respx
2	Write GitHubClient class in client.py with base_url, default headers (auth, version)
3	Implement get(url) -> GitHubResponse with body, status_code, rate_limit, etag fields
4	Implement paginate(url) -> AsyncIterator that follows Link: rel=next
5	Write RateLimiter class - reads X-RateLimit-Remaining, logs warning below threshold
6	Implement exponential backoff for 429 / 403 rate limit responses
7	Write GitHubRepository service: get_pr(owner, repo, number) -> PRSummary
8	Write get_pr_diff(owner, repo, number) -> str (raw unified diff)
9	Write get_pr_files(owner, repo, number) -> list[PRFile]
10	Extend core/cache.py with cache_key builders and cache_aside() helper
11	Wrap every GitHub call with cache-aside (check Redis first, store on miss)
12	Implement ETag flow: store ETag in Redis, send If-None-Match, handle 304
13	Capture a real PR response from your test repo, save as fixture JSON
14	Write mocked tests using respx for all 3 repository methods
15	Verify: second call to get_pr hits Redis not GitHub (check logs)

Definition of Done

[OK] get_pr() returns PRSummary for a real test PR
[OK] get_pr_diff() returns raw unified diff string
[OK] Second call for same PR shows cache_hit=True in logs
[OK] Rate limit remaining is logged on every GitHub call
[OK] All tests pass with respx mocks (no real GitHub calls in CI)

Verify with: pytest tests/test_github_client.py -v

Phase 3 - Semantic Diff Engine (Days 5-6)

Next

Objective

Build the intellectual core of ReviewMind: parse raw unified diffs into structured SemanticChunks using the unidiff library + Python AST module. Each chunk knows its file, line range, containing function/class, change type, and complexity score.

Files to Create

- reviewmind/engine/__init__.py
- reviewmind/engine/models.py - SemanticChunk, FileDiff, Hunk Pydantic models
- reviewmind/engine/diff_parser.py - unidiff-based parser -> FileDiff + Hunk objects
- reviewmind/engine/chunker.py - AST mapping of hunks to functions/classes
- reviewmind/engine/classifier.py - new_feature/bug_fix/refactor/test/config/dep
- reviewmind/engine/complexity.py - cyclomatic complexity per function
- reviewmind/services/analysis/
- reviewmind/services/analysis/from_paste.py - code string -> CodeInput
- reviewmind/services/analysis/from_upload.py - file list -> CodeInput
- tests/fixtures/diffs/ - .diff fixture files
- tests/test_diff_parser.py
- tests/test_chunker.py

Step-by-Step Build Tasks

1	pip install unidiff
2	Define SemanticChunk model: file_path, start_line, end_line, chunk_type, classification, complexity, content, diff
3	Write diff_parser.py: parse raw diff string -> list[FileDiff] using unidiff library
4	Each FileDiff: filename, change_type (added/modified/deleted/renamed), list of Hunks
5	Each Hunk: old_start, old_length, new_start, new_length, lines[(type, content)]
6	Write chunker.py: for each changed file, fetch full file from GitHub, run ast.parse()
7	Map hunk line ranges to enclosing FunctionDef / ClassDef AST nodes
8	Merge hunks in the same function into one SemanticChunk
9	Handle module-level code (no enclosing function) -> chunk_type='module'
10	Write classifier.py: classify each chunk (new_feature if function didn't exist before, etc.)
11	Write complexity.py: count if/elif/for/while/except/and/or -> complexity = 1 + count
12	Write from_paste.py and from_upload.py adapters -> CodeInput(source, files)
13	Collect 3 fixture .diff files: one with new function, one bug fix, one config change
14	Write tests: given fixture diff -> correct chunk count, types, line ranges, complexity

Definition of Done

[OK] parse_diff(fixture) returns correct FileDiff list with right hunk boundaries
[OK] chunker maps a multi-hunk function correctly to one SemanticChunk
[OK] classifier correctly labels new function vs modified function
[OK] complexity score matches manual count for 3 hand-checked examples
[OK] 90%+ test coverage on diff_parser.py and chunker.py

Verify with: `pytest tests/test_diff_parser.py tests/test_chunker.py -v --cov=reviewmind/engine`

Phase 4 - Security Analysis Engine (Day 7)

Upcoming

Objective

Implement the 5 security rules as a pluggable rule registry. Each rule is a small function taking a SemanticChunk and returning SecurityFinding objects. Rules run against all chunks from Phase 3.

Files to Create

- reviewmind/engine/security/__init__.py
- reviewmind/engine/security/models.py - SecurityFinding Pydantic model
- reviewmind/engine/security/rules.py - SEC001 through SEC005 rule functions
- reviewmind/engine/security/engine.py - rule registry, iterate + aggregate
- tests/test_security_rules.py - 3 test cases per rule

Step-by-Step Build Tasks

1	Define SecurityFinding: rule_id, severity, file_path, line, message, snippet
2	Write rule registry: list of (rule_id, fn) pairs - engine iterates all rules per chunk
3	SEC001 Hardcoded Secret: regex on added lines for AWS keys, api_key=, BEGIN PRIVATE KEY
4	SEC001: allowlist for test/example/sample files (lower severity)
5	SEC002 SQL Injection: detect f-string/format/% in cursor.execute() / .raw() args
6	SEC002: confirm parameterized queries are NOT flagged
7	SEC003 Insecure Dep: parse requirements.txt/pyproject.toml added lines vs vuln snapshot JSON
8	SEC004 Missing Validation: flag route handlers using path params in open() or execute()
9	SEC005 Unsafe Deserialization: flag pickle.load, yaml.load without SafeLoader, eval, exec
10	Write engine.py: run all enabled rules per chunk, deduplicate, sort by severity
11	Write test_security_rules.py: for each rule - one positive (catches it), one negative (ignores safe), one false-positive (documents known FP)

Definition of Done

[OK]Each SEC rule catches its target pattern in fixture code
[OK]Parameterized SQL query is NOT flagged by SEC002
[OK]yaml.safe_load() is NOT flagged by SEC005
[OK]3+ test cases per rule passing
[OK]90%+ coverage on rules.py

Verify with: pytest tests/test_security_rules.py -v --cov=reviewmind/engine/security

Phase 5 - Style Engine + reviewmind.yaml (Day 8)

Upcoming

Objective

Build the style analysis engine driven by a per-repo reviewmind.yaml config file. Rules are togglable per project. Six built-in rules covering function length, naming conventions, forbidden imports, line length, docstrings, and TODO tickets.

Files to Create

- reviewmind/engine/style/__init__.py
- reviewmind/engine/style/models.py - StyleFinding, RuleConfig Pydantic models
- reviewmind/engine/style/rules.py - STY001 through STY006 rule functions
- reviewmind/engine/style/engine.py - load reviewmind.yaml, filter + run rules
- reviewmind.yaml - example config for this repo
- tests/test_style_rules.py

Step-by-Step Build Tasks

1	Define RuleConfig Pydantic model matching reviewmind.yaml schema
2	Write YAML loader: fetch reviewmind.yaml from repo root via GitHub client, parse with PyYAML
3	Cache parsed config in Redis (repo config rarely changes - 1h TTL)
4	STY001 Max Function Length: count lines in chunk.content vs config.max_function_length
5	STY002 Missing Docstring: check AST for FunctionDef / ClassDef missing docstring node
6	STY003 Naming Convention: regex match per identifier type (snake_case / PascalCase)
7	STY004 Forbidden Import: check added lines for patterns in config.forbidden_imports list
8	STY005 Max Line Length: check each added line length vs config.max_line_length
9	STY006 TODO without ticket: regex for TODO/FIXME not followed by #123 or JIRA-456
10	Write engine.py: load config, filter rules by enabled flag, run per chunk
11	Write example reviewmind.yaml with all 6 rules configured
12	Write tests: toggling a rule off in config removes findings for that rule

Definition of Done

[OK]Disabling STY001 in reviewmind.yaml removes all max_function_length findings

[OK]PascalCase function name is flagged by STY003

[OK]Missing docstring finding appears for public functions

[OK]Rules correctly skip test files where chunk_type='test'

Verify with: pytest tests/test_style_rules.py -v --cov=reviewmind/engine/style

Phase 6 - MCP Tools + Claude API Loop (Days 9-10)

Upcoming

Objective

Wire all 19 MCP tools to the Service Layer. Add a /chat FastAPI endpoint that runs the Claude API tool loop: send message + tools, execute returned tool calls via MCP, send results back, repeat until Claude stops calling tools.

Files to Create

- reviewmind/mcp_server.py - add all 19 real tools (replace ping stub)
- reviewmind/api/routes.py - add POST /chat endpoint
- reviewmind/services/review.py - post_review service (GitHub Reviews API)
- reviewmind/services/fix.py - apply_fix service (code patching logic)
- reviewmind/api/auth.py - JWT auth middleware for gated routes
- tests/test_mcp_tools.py - tools/list + tools/call per tool
- tests/test_e2e_golden_path.py - full flow: fixture PR -> findings -> review
- claude_desktop_config.json - (local, not committed) for manual testing

Step-by-Step Build Tasks

1	pip install anthropic python-jose[cryptography]
2	Register all 19 tools in mcp_server.py with correct inputSchema JSON schemas
3	Each tool handler: validate args -> call service function -> format MCP result
4	Write /chat endpoint in routes.py: accepts {message, code?, files?}
5	/chat endpoint: get tools from MCP via tools/list, send to Claude API
6	Implement tool loop: while stop_reason == tool_use: call tool via MCP, append result, re-send
7	Write auth.py: JWT decode middleware, require_auth dependency for apply_fix + post_review
8	Add /auth/login endpoint: validate credentials -> return JWT
9	Write apply_fix service: take finding + code -> apply patch -> return modified code
10	Write post_review service: POST to GitHub Reviews API, persist to reviews table
11	Test each tool via tools/list and tools/call directly (no Claude Desktop needed)
12	Write e2e golden path test: fixture diff -> analyze -> security -> style -> review draft
13	Manually verify in Claude Desktop: 'Review PR #X on owner/repo' works end-to-end

Definition of Done

[OK] tools/list returns all 19 tools with correct schemas

[OK] GET /health + POST /chat both respond correctly

[OK] Full golden path works: user message -> Claude -> tools -> synthesized review

[OK] apply_fix is blocked without JWT, succeeds with valid JWT

[OK] e2e golden path test passes with fixture data

Verify with: pytest tests/test_mcp_tools.py tests/test_e2e_golden_path.py -v

Phase 7 - Frontend (React / Next.js) (Days 11-14)

Upcoming

Objective

Build the user-facing frontend: a chat interface where users can type messages, paste code, upload files, and see findings displayed with syntax highlighting. Auth flow for apply_fix and post_review. All API calls go to the FastAPI backend.

Files to Create

- frontend/ - new Next.js project
- frontend/app/page.tsx - main chat UI
- frontend/components/ChatInput.tsx - message input + code paste + file upload
- frontend/components/FindingCard.tsx- individual finding with severity badge
- frontend/components/CodeDiff.tsx - before/after diff viewer for apply_fix
- frontend/components/AuthModal.tsx - login modal triggered by auth gate
- frontend/lib/api.ts - typed API client for FastAPI backend

Step-by-Step Build Tasks

1	npx create-next-app frontend --typescript --tailwind
2	Build ChatInput: text area for messages, code paste box, file drag-drop zone
3	Build message list: user messages + Claude responses with markdown rendering
4	Build FindingCard: rule_id badge, severity color, file+line, message, 'Suggest Fix' button
5	Wire 'Suggest Fix' button -> POST /chat {message: 'fix finding X'} -> shows CodeDiff
6	Build CodeDiff: side-by-side original vs fixed code using react-diff-viewer
7	Wire 'Apply Fix' button -> auth check -> show AuthModal if not logged in
8	Build AuthModal: email/password form -> POST /auth/login -> store JWT in memory
9	After auth: retry apply_fix with JWT -> show success + copy-to-clipboard button
10	File upload: input[type=file] -> read as text -> include in POST /chat as files[]
11	Add per-file tabs when multiple files uploaded - one FindingCard list per file
12	Test golden path in browser: paste SQL injection code -> see SEC002 finding -> suggest fix -> login -> apply

Definition of Done

[OK]User can paste code and see findings without logging in

[OK]Clicking Apply Fix triggers login modal if not authenticated

[OK]After login, apply_fix returns patched code shown in CodeDiff

[OK]File upload works for .py, .ts, .yaml files

[OK]PR review flow works end-to-end from chat input to displayed review

Verify with: cd frontend && npm run dev # then test manually in browser

Phase 8 - A2A Parallel Agents (Post-MVP)

Future

Objective

Introduce Agent-to-Agent (A2A) communication so the security, style, and diff engines run as independent parallel sub-agents instead of sequentially. ReviewMind becomes an orchestrator, coordinating specialist agents and merging results.

Files to Create

- reviewmind/agents/__init__.py
- reviewmind/agents/orchestrator.py - coordinates sub-agents, merges results
- reviewmind/agents/security_agent.py - wraps security engine as A2A agent
- reviewmind/agents/style_agent.py - wraps style engine as A2A agent
- reviewmind/agents/diff_agent.py - wraps diff engine as A2A agent

Step-by-Step Build Tasks

1	Define A2A agent interface: agent card (name, capabilities), task endpoint
2	Wrap SecurityEngine as a standalone A2A agent (its own FastAPI sub-app)
3	Wrap StyleEngine as a standalone A2A agent
4	Write Orchestrator: dispatch tasks to all 3 agents concurrently (asyncio.gather)
5	Merge results: combine SecurityFindings + StyleFindings + SemanticChunks
6	Expose ReviewMind itself as an A2A agent so other systems can call it
7	Benchmark: measure latency improvement vs sequential execution on a large PR

Definition of Done

[OK]Security, style, diff analysis run concurrently (measured via logs)
[OK]Orchestrator correctly merges findings from all 3 agents
[OK]ReviewMind responds to A2A agent discovery requests
[OK]Latency on a 20-file PR is < 50% of sequential baseline

Verify with: pytest tests/test_agents.py -v

What to Build Next - Phase 3 Action Plan

Phases 1 and 2 are complete. Phase 3 is the intellectual core of ReviewMind: parsing raw unified diffs into structured SemanticChunks using unidiff + Python AST. Follow the tasks in order - each one builds on the last.

Before You Start Phase 3

- Confirm Phase 2 tests pass: pytest tests/test_github_client.py -v -> 12 passed
- Install new dependency: pip install unidiff
- Collect 3 fixture .diff files: one new function, one bug fix, one config change
- Understand Python ast module: ast.parse(), ast.walk(), FunctionDef, ClassDef nodes
- Read unidiff docs: PatchSet, PatchedFile, Hunk, Line (is_added / is_removed / is_context)

Phase 3 Task Checklist (Days 5-6)

[ ]	**pip install unidiff && add to pyproject.toml dependencies** *unidiff library parses unified diff strings into PatchSet -> PatchedFile -> Hunk -> Line objects*
[ ]	**Create reviewmind/engine/__init__.py** *Empty file - marks engine/ as a package*
[ ]	**Write engine/models.py - SemanticChunk Pydantic model** *Fields: file_path, start_line, end_line, chunk_type (function/class/module), classification, complexity, content, diff_lines*
[ ]	**Write engine/models.py - FileDiff + Hunk models** *FileDiff: filename, change_type (added/modified/deleted/renamed), hunks[]. Hunk: old_start, new_start, lines[(type, content)]*
[ ]	**Write engine/diff_parser.py - parse_diff(raw_diff: str) -> list[FileDiff]** *Use unidiff.PatchSet.from_string(raw_diff). Map PatchedFile -> FileDiff, Hunk -> Hunk model*
[ ]	**Detect change_type from PatchedFile.is_added_file / is_removed_file / is_rename** *Renamed files: PatchedFile.source_file != PatchedFile.target_file*
[ ]	**Write engine/chunker.py - map hunks to AST nodes** *For each changed .py file: fetch full source via GitHubRepository.get_file_content(), run ast.parse()*
[ ]	**Walk AST to find FunctionDef/AsyncFunctionDef/ClassDef per changed line range** *ast.walk() + node.lineno/end_lineno. If hunk lines overlap with node range -> assign chunk to that node*
[ ]	**Merge hunks that touch the same function into one SemanticChunk** *Group by (file_path, function_name). One chunk per function even if 3 separate hunks touch it*
[ ]	**Handle module-level code with no enclosing function** *chunk_type='module', name='<module>'. Use this when no FunctionDef/ClassDef contains the changed lines*
[ ]	**Write engine/classifier.py - classify each chunk** *new_feature: function did not exist in old file (added). bug_fix: modified + file has 'fix'/'bug' in name or commit msg. refactor: modified, no logic change (rename/format). test: file*
[ ]	**Write engine/complexity.py - cyclomatic complexity per function** *complexity = 1 + count of: if, elif, for, while, except, and, or, with keywords in chunk.content*
[ ]	**Write services/analysis/from_paste.py** *Takes a code string + optional filename -> wraps as CodeInput(source='paste', files=[{filename, content}])*
[ ]	**Write services/analysis/from_upload.py** *Takes list of (filename, bytes) -> decodes -> returns CodeInput(source='upload', files=[...])*
[ ]	**Collect fixture diffs in tests/fixtures/diffs/** *new_function.diff, bug_fix.diff, config_change.diff - use real diffs from your GitHub test repo*
[ ]	**Write tests/test_diff_parser.py** *Given fixture diff -> assert correct FileDiff count, hunk boundaries, added/removed line counts*
[ ]	**Write tests/test_chunker.py** *Given fixture diff + mocked file content -> assert SemanticChunks with correct line ranges, function names, chunk_types*
[ ]	**Verify complexity score on 3 hand-checked examples** *Count branches manually, compare to complexity() output. Off-by-one errors are common here*

Tip: non-Python files (JS, YAML, etc.) skip AST chunking - use line-range chunks only.

Only .py files get AST mapping. For all other extensions, one SemanticChunk per hunk with chunk_type='raw'.